LAB 21 – Collaborative AI Coding: Pair Programming and Version Control Integration

NAME:T.SATHVIK

ROLL-NO:2503A52415

TASK1:

Prompt given to AI:

"Generate a simple Python To-Do List Manager with options to add tasks, view tasks, delete tasks and exit. Then improve the program by adding error handling, input validation and a feature to mark tasks as completed.

Old Code:

```
tasks = []
while True:
    print("\n===== TO-DO LIST MANAGER =====")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Delete Task")
    print("4. Exit")
```

```
    choice = input("Enter your choice: ")

    if choice == "1":
        task = input("Enter task: ")
        tasks.append(task)
        print("Task added successfully!")

    elif choice == "2":
        for i, task in enumerate(tasks, start=1):
            print(i, ".", task)

    elif choice == "3":
        task_no = int(input("Enter task number: "))
        tasks.pop(task_no - 1)
        print("Task deleted successfully!")

    elif choice == "4":
        break

    else:
        print("Invalid choice!")
```

IMPROVED CODE:

```
tasks = []

while True:
    print("\n===== TO-DO LIST MANAGER =====")
```

```
print("1. Add Task")
print("2. View Tasks")
print("3. Delete Task")
print("4. Exit")

choice = input("Enter your choice: ")

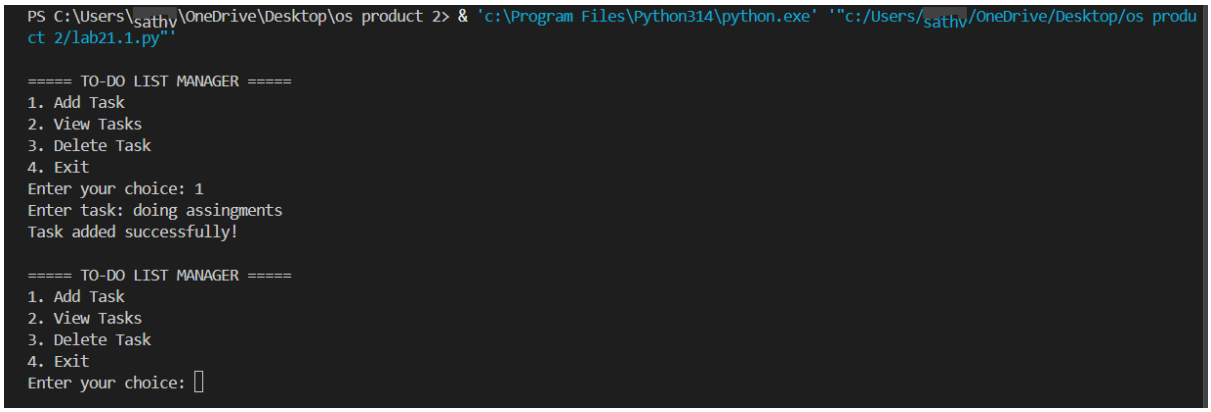
if choice == "1":
    task = input("Enter task: ")
    tasks.append(task)
    print("Task added successfully!")

elif choice == "2":
    for i, task in enumerate(tasks, start=1):
        print(i, ".", task)

elif choice == "3":
    if tasks:
        task_no = int(input("Enter task number: "))
        if 1 <= task_no <= len(tasks):
            tasks.pop(task_no - 1)
            print("Task deleted successfully!")
        else:
            print("Invalid task number!")
    else:
        print("No tasks to delete!")
```

```
    elif choice == "4":
        break
    print("Invalid choice!")
```

OUTPUT:



TASK2:

Prompt given to AI:

"Explain how to initialize a Git repository, create and switch to a new branch, add AI-assisted code modifications, commit changes with a meaningful message and merge the changes into the main branch. Provide the Git commands with explanations."

Old Code:

```
tasks = []


while True:
    print("\n===== TO-DO LIST MANAGER =====")
    print("1. Add Task")
    print("2. View Tasks")
```

```
    print("3. Delete Task")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        task = input("Enter task: ")
        tasks.append(task)
        print("Task added successfully!")

    elif choice == "2":
        for i, task in enumerate(tasks, start=1):
            print(i, ".", task)

    elif choice == "3":
        task_no = int(input("Enter task number: "))
        tasks.pop(task_no - 1)
        print("Task deleted successfully!")

    elif choice == "4":
        break

    else:
        print("Invalid choice!")
```

IMPROVED CODE:

```
tasks = []
```

```
def add_task():
    task = input("Enter task: ").strip()

    if task:
        tasks.append({"name": task, "completed": False})
        print("Task added successfully!")
    else:
        print("Task cannot be empty!")



def view_tasks():
    if not tasks:
        print("No tasks available!")
        return

    for i, task in enumerate(tasks, start=1):
        status = "Completed" if task["completed"] else "Pending"
        print(i, task["name"], "-", status)



def delete_task():
    view_tasks()

    if not tasks:
        return
```

```
    try:
        n = int(input("Enter task number: "))

        if 1 <= n <= len(tasks):
            tasks.pop(n - 1)
            print("Task deleted successfully!")
        else:
            print("Invalid task number!")

    except ValueError:
        print("Enter a valid number!")



def complete_task():
    view_tasks()

    if not tasks:
        return

    try:
        n = int(input("Enter task number: "))

        if 1 <= n <= len(tasks):
            tasks[n-1]["completed"] = True
            print("Task completed!")
```

```
            else:
                print("Invalid task number!")

        except ValueError:
            print("Enter a valid number!")



while True:
    print("\n===== TO-DO LIST MANAGER =====")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Delete Task")
    print("4. Complete Task")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        add_task()
    elif choice == "2":
        view_tasks()
    elif choice == "3":
        delete_task()
    elif choice == "4":
        complete_task()
    elif choice == "5":
```

```
        break
    else:
        print("Invalid choice!")
```

OUTPUT:



Git Commands:

```
git init
git add .
git commit -m "Add original todo manager"

git branch feature/improvements
git switch feature/improvements

git add .
git commit -m "Add AI-assisted improvements"

git switch main
git merge feature/improvements

git log --oneline --all --graph
```

TASK3:

Prompt given to AI:

"Explain how to initialize a Git repository, create and switch to a new branch, add AI-assisted code modifications, commit changes with a meaningful

message and merge the changes into the main branch. Provide the Git commands with explanations."

Old Code:

```
tasks = []

task = input("Enter task: ")
tasks.append(task)

print("Tasks:", tasks)

task_no = int(input("Enter task number to delete: "))
tasks.pop(task_no - 1)

print("Updated tasks:", tasks)
```

IMPROVED CODE:

```
tasks = []

def add_task():
    task = input("Enter task: ").strip()

    if task:
        tasks.append({"name": task, "completed": False})
        print("Task added")
    else:
```

```
        print("Task cannot be empty")

def view_tasks():
    for i, task in enumerate(tasks, start=1):
        status = "Completed" if task["completed"] else "Pending"
        print(i, task["name"], "-", status)

def complete_task():
    view_tasks()

    try:
        number = int(input("Enter task number: "))

        if 1 <= number <= len(tasks):
            tasks[number - 1]["completed"] = True
            print("Task completed")
        else:
            print("Invalid task number")

    except ValueError:
        print("Enter a valid number")

add_task()
view_tasks()
complete_task()
view_tasks()
```

OUTPUT:

```
PS C:\Users\sathv\OneDrive\Desktop\ai assisted coding> & 'c:\Program Files\Python314\python.exe' '"c:/Users/sathv/OneDrive/Desktop/ai assisted coding/lab21.3.py"'
Enter task: 2
Task added
1 2 - Pending
1 2 - Pending
Enter task number: 5
Invalid task number
1 2 - Pending
PS C:\Users\sathv\OneDrive\Desktop\ai assisted coding>
```

Git Commands:

```
git add .
git commit -m "Improve todo manager with validation and task completion"

git log --oneline
```

TASK4:

Prompt given to AI:

"Create a simple Python Calculator program. Student A writes the basic calculator with addition, subtraction, multiplication and division. Student B uses AI assistance to improve the program by adding input validation, error handling and better code organization using functions. Explain how both students can collaborate using Git branches and merging."

Old Code:

```
num1 = int(input("Enter first number: "))

num2 = int(input("Enter second number: "))


print("1. Addition")

print("2. Subtraction")

print("3. Multiplication")

print("4. Division")


choice = int(input("Enter choice: "))
```

```
if choice == 1:
    print("Result:", num1 + num2)
elif choice == 2:
    print("Result:", num1 - num2)
elif choice == 3:
    print("Result:", num1 * num2)
elif choice == 4:
    print("Result:", num1 / num2)
else:
    print("Invalid choice")
```

IMPROVED CODE:

```
def calculator():
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))

        print("\n1. Addition")
        print("2. Subtraction")
        print("3. Multiplication")
        print("4. Division")

        choice = input("Enter choice: ")

        if choice == "1":
```

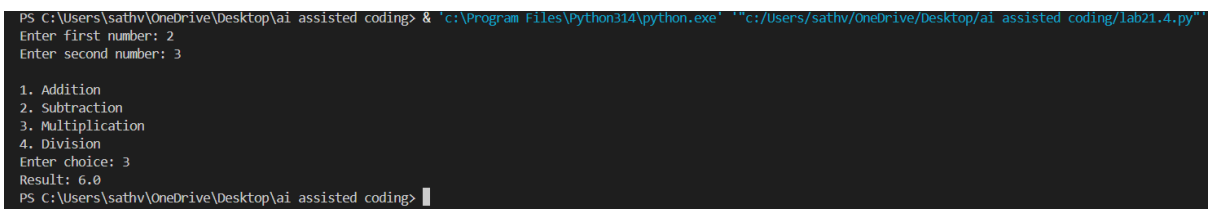
```
            result = num1 + num2
        elif choice == "2":
            result = num1 - num2
        elif choice == "3":
            result = num1 * num2
        elif choice == "4":
            if num2 == 0:
                print("Cannot divide by zero")
                return
            result = num1 / num2
        else:
            print("Invalid choice")
            return

        print("Result:", result)

    except ValueError:
        print("Please enter valid numbers")

calculator()
```

OUTPUT:



Git Commands:

```
Initialized empty Git repository

[main] Add basic calculator

Switched to branch 'student-b-improvements'

[student-b-improvements] Improve calculator with validation and error handling

Switched to branch 'main'

Updating main
Fast-forward
Merge successful
```

TASK5:

Prompt given to AI:

"Find and fix the syntax and logical errors in the following Python program. Explain the errors and provide the corrected code."

Old Code:

```
# Program to calculate the average of three numbers


num1 = input("Enter first number: ")

num2 = input("Enter second number: ")

num3 = input("Enter third number: ")


total = num1 + num2 + num3

average = total / 2


if average > 50

    print("Pass")

else:
```

```
    print("Fail")
```

IMPROVED CODE:

```
# Program to calculate the average of three numbers


try:
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))
    num3 = float(input("Enter third number: "))

    total = num1 + num2 + num3
    average = total / 3

    print("Total:", total)
    print("Average:", average)

    if average >= 50:
        print("Pass")
    else:
        print("Fail")

except ValueError:
    print("Please enter valid numbers")
```

OUTPUT:



COMPARISION:

Buggy Version	Corrected Version
Missing colon after if	Added the missing colon
Used input() without converting to numbers	Used float()
Calculated average by dividing by 2	Corrected division to 3
No invalid-input handling	Added try-except
Incorrect pass condition (> 50)	Uses >= 50

TASK6:

Prompt given to AI:

"Create a basic Python calculator with addition, subtraction, multiplication and division. Use AI to improve it by adding calculation history, input validation, division-by-zero handling and better code organization."

Old Code:

```
num1 = float(input("Enter first number: "))

num2 = float(input("Enter second number: "))


print("1. Addition")

print("2. Subtraction")
```

```
print("3. Multiplication")
print("4. Division")

choice = int(input("Enter choice: "))

if choice == 1:
    print("Result:", num1 + num2)
elif choice == 2:
    print("Result:", num1 - num2)
elif choice == 3:
    print("Result:", num1 * num2)
elif choice == 4:
    print("Result:", num1 / num2)
else:
    print("Invalid choice")
```

IMPROVED CODE:

```
history = []

def calculator():
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))

        print("\n1. Addition")
        print("2. Subtraction")
        print("3. Multiplication")
```

```
print("4. Division")

choice = input("Enter choice: ")

if choice == "1":
    result = num1 + num2
    operator = "+"
elif choice == "2":
    result = num1 - num2
    operator = "-"
elif choice == "3":
    result = num1 * num2
    operator = "*"
elif choice == "4":
    if num2 == 0:
        print("Cannot divide by zero")
        return
    result = num1 / num2
    operator = "/"
else:
    print("Invalid choice")
    return

print("Result:", result)
history.append(f"{num1} {operator} {num2} = {result}")
```

```
    except ValueError:
        print("Please enter valid numbers")

def show_history():
    if not history:
        print("No calculation history")
    else:
        print("\nCalculation History:")
        for item in history:
            print(item)

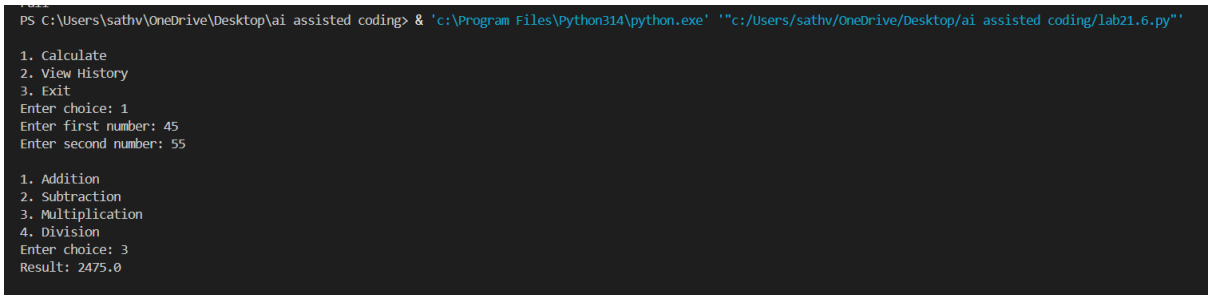
while True:
    print("\n1. Calculate")
    print("2. View History")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        calculator()
    elif choice == "2":
        show_history()
    elif choice == "3":
        print("Exiting calculator")
        break
    else:
```

```
print("Invalid choice")
```

OUTPUT:



Analysis of Improvements

Basic Calculator	AI-Enhanced Calculator
Performs basic calculations	Performs calculations and stores history
No input validation	Handles invalid inputs
No division-by-zero protection	Prevents division by zero
No history feature	Displays previous calculations
Limited functionality	More user-friendly and reliable